Column Stores

- Column-Oriented Databases Management Systems (CODBMS) stores data using tables, rows, and columns.
- Unlike a relational database, the names and format of the columns can vary from row to row in the same table.
- Columns are grouped in Column Families.
- CODBMSs are also called Wide Column stores or Columnar Database.

Column Stores

Column Families

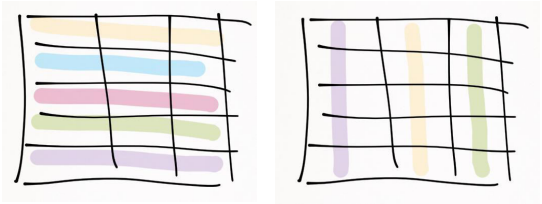
row key	personal data		professional data	
employee ID	Name	City	Title	Salary
1342	Joseph	New York City	Product Manager	$150,000
1543	Frank	Boston	Software Engineer	$160,000
7643	David	San Francisco	Data Scientist	$130,000

Column Stores

- CODBMS Databases uses one of two approaches to partitioning data on disk:
  - Row-Oriented: partition tables by rows
  - Column-Oriented: partition tables by columns/column families

Column-Oriented Databases

- Partitioning: Row-Oriented vs. Column-Oriented



Column Stores

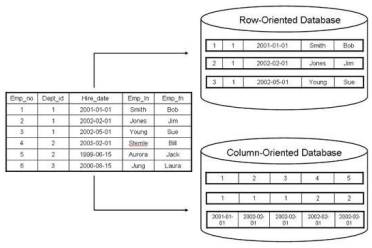
- CODBMS disk storage approach: Row-Oriented partitioning
  - Predictability:
    - Tuples address can be predicted without looking at the data
  - Locality:
    - Reduce the probability to read the data from memory cache or the same block on disk
  - Read operations:
    - To read one attribute the entire tuple must be read

Column Stores

- CODBMS disk storage approach: Column-Oriented partitioning
  - Predictability:
    - We can predict exactly where we find the data if the data uses fixed length types
    - We cannot predict exactly where we find the data in the case of tuples with variable length
  - Locality:
    - The probability to hit the cache (or block) is very high during iteration
  - Read operations:
    - Read only the data needed

Column Stores

- Partitioning: Row-Oriented vs. Column-Oriented



Cassandra

- Cassandra
  - First developed by Facebook to solve the inbox search problem
  - The team wanted to address the following problems:
    - Handle huge volumes of data in the form of message copies
    - Reverse indices of messages
    - Many random reads
    - Many simultaneous random writes

Cassandra

- Cassandra
  - Is open source
  - Is Distributed and Decentralized
  - Is Elastically Scalable
  - Is Highly Available and Fault-Tolerant
  - Uses Tunable Consistency
  - High Performance
  - Row-Oriented Partitioning

Cassandra

- Cassandra is distributed
  - Cassandra can run on multiple machines while appearing to users as a unified whole
  - Cassandra design and code base is specifically engineered to:
    - Work across many different machines
    - Optimize performance across multiple data center racks
    - Run a single Cassandra clusters across geographically dispersed data centers

Cassandra

- Cassandra is decentralized
  - There is no single point of failure
  - All the nodes in a Cassandra cluster function exactly the same
    - This is sometimes referred to as “server symmetry”
  - Because they are all doing the same thing there can’t be a special host that is coordinating activities, as with the master/slave setup

Cassandra

- Cassandra is Elastic Scalable
  - Elastic scalability refers to a special property of horizontal scalability
  - It means that your cluster can seamlessly
    - Scale up
    - Scale (back) down
  - The cluster must be able:
    - To seamlessly accept new nodes
    - New nodes can begin participating by getting a copy of some or all the data
    - New nodes start serving new user requests
      - Without major disruption
      - Reconfiguration of the entire cluster.

Cassandra

- With Elastic Scalability, there is no need to:
  - Restart any process
  - Change any application queries
  - Manually rebalance the data
- With Elastic Scalability, Cassandra will automatically find, start and send work to any new machine added to the cluster

Cassandra

- Cassandra is Highly Available and Fault Tolerance
  - Failed nodes can be replaced in the cluster with no downtime
  - Data can be replicated to multiple data centers to offer improved local performance
  - Replication helps to prevent downtime if one data center experiences a catastrophe

Cassandra

- Cassandra uses Tunable Consistency
  - Cassandra allows the administrator to choose the level of consistency
  - Cassandra’s consistency also uses the replication factor
  - The replication factor is a trade-off between how much you want to pay in performance to gain more consistency
  - The replication factor value is equal to the number of nodes in the cluster where a modification (insert, update, delete) will propagate to

Cassandra

- Cassandra consistency level
  - Is a setting that clients must specify on every operation
  - The consistency level allows clients to decide how many replicas in the cluster
    - Must acknowledge a write operation in order to be considered consistent and durable
    - Respond to a read operation in order to be considered successful
  - Can be equal to the replication factor:
    - The gain is stronger consistency
    - The cost is synchronous blocking operations that wait for all nodes to be updated and declare success before returning

Cassandra

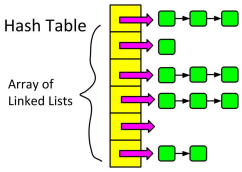
- Cassandra is a High-Performance database
  - Cassandra was designed specifically from the ground up to take full advantage of multiprocessor/multi-core machines
  - It can run across many dozens of these machines housed in multiple data centers
  - It scales consistently and seamlessly to hundreds of terabytes
  - Cassandra has been shown to perform exceptionally well under heavy load

Cassandra

- Cassandra is a High-Performance database
  - It consistently can show very fast throughput for writes per second on basic commodity computers
  - The commodity computers can run on
    - Physical hardware
    - Virtual machines
    - Containers
  - All of Cassandra's desirable properties can be maintained without sacrificing performance as more servers are added

Cassandra

- Cassandra's Row-Oriented partitioning
  - Cassandra's data model can be described as a partitioned row store
  - Data is stored in sparse multidimensional hashtables.



Cassandra

- Cassandra's Row-Oriented partitioning
  - Sparse means that
    - For any given row you can have one or more columns
    - Each row does not need to have all the same columns as other rows like it (as in a relational model)
  - Partitioned means that
    - Each row has a unique key which makes its data accessible
    - The keys are used to distribute the rows across multiple data stores.

Cassandra

row key	columns...			
123	1339707619:"value"	1339817619:"value"	1369707619:"value"	
	1.1	1.11	0.87	
456	1339707619:"value"	1339817619:"value"	1369707619:"value"	1369707888:"value"
	40	111	144	222
789	1339707619:"value"			
	21.345			

Cassandra

- Cassandra is a good fit for
  - Large Deployments
  - Lots of Writes, Statistics, and Analytics
  - Geographical Distribution
  - Evolving Applications

Cassandra

- Large Deployments:
  - If the application is expected to require dozens of nodes
  - For applications that needs high availability
  - For applications that are geographical distributed

Cassandra

- Lots of Writes, Statistics, and Analytics
  - Before choosing a database, consider your application from the perspective of the ratio of reads to writes
  - Cassandra is optimized for excellent throughput on writes
  - One of Cassandra's primary feature is the ability to handle application workloads that require high performance at significant write volumes with many concurrent client threads

Cassandra

- Lots of Writes, Statistics, and Analytics
  - Cassandra is a good choice for storing
    - User activity updates
    - Social network usage
    - Recommendations
    - Reviews
    - Application statistics
  - Cassandra is also a good choice when the application involve lots of writing with less predictable read operations
  - When updates can occur unevenly with sudden spikes

Cassandra

- Lots of Writes, Statistics, and Analytics
  - Cassandra has been used to create a variety of applications:
    - Windowed time-series store
    - An inverted index for document searching
    - A distributed job priority queue

Cassandra

- Geographical Distribution
  - Cassandra has out-of-the-box support for geographical distribution of data
  - Cassandra can be easily configured to replicate data across multiple data centers
  - Cassandra could be a great fit if you have a globally deployed application that could see a performance benefit from putting the data near the user

Cassandra

- Evolving Applications
  - Cassandra might be a good fit if the application is evolving rapidly given its support for flexible schemas
  - Cassandra makes it easy to keep your database in step with application changes as you rapidly deploy

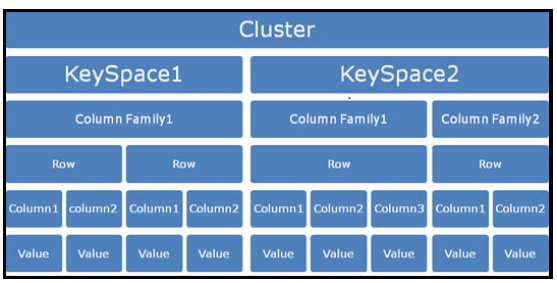
Cassandra – Data Model

- Cassandra data model contains the following structures:
  - Cluster
  - Keyspace
  - Table

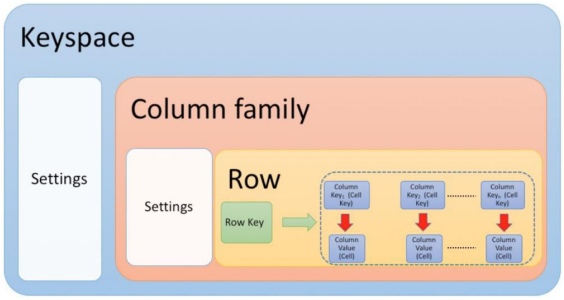
Cassandra – Data Model

- An instance of Cassandra typically consists of only one table
- A table represents a distributed multidimensional map indexed by a key.
- A table is structured by the following dimensions (bottom-up view)
  - Rows
  - Row Key (Partition Key)
  - Column
  - Super Column
  - Column Family
  - Super Column Family

Cassandra – Data Model



Cassandra – Data Model



Cassandra – Data Model

- Rows
  - A row is the smallest unit that stores related data in Cassandra
  - Rows are identified by a string-key of arbitrary length
  - Operations on rows are atomic per replica no matter how many columns are being read or written

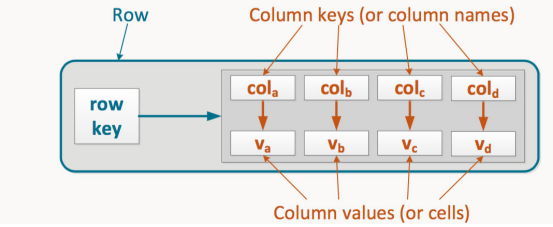
Cassandra – Data Model

- Rows
  - A row key uniquely identifies a row in a column family
  - A row stores pairs of column keys and column values
  - A column key uniquely identifies a column value in a row
  - A column value stores one value or a collection of values

Cassandra – Data Model

- Rows may be described as skinny or wide
  - Skinny row:
    - Has a fixed number of column keys
    - Relatively small number of column keys.
  - Wide row:
    - Has a relatively large number of column keys (hundreds or thousands)
    - The number of column keys may increase as new data values are inserted

Cassandra – Data Model



Cassandra – Data Model

- Keys
  - Composite column key:
    - Multiple components can form a column key
    - The components are separated by colon



Cassandra – Data Model

- Column
  - Stores a values
  - Is identified by a timestamp and a name
  - The name acts as the column key
  - Each row in a table can have a different number of columns:
    - A table can be thought of as a rectangle.

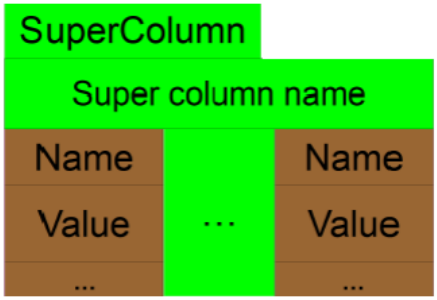
Cassandra – Data Model

- Columns
  - Client applications may specify the ordering of columns within
    - Super column
    - Column Family
  - The ordering can either be by name or by timestamp

Cassandra – Data Model

- Super columns
  - Are column containers sorted by their names
  - Are tuples (pairs) that consist of a key-value pair
    - The key is mapped to a value that is a set of columns
    - Can have an arbitrary number of columns associated with them
  - The number of columns per super column may differ per row
  - Must have a name
  - The name acts as a unique identifier
  - Can be referenced and sorted by their row key

Cassandra – Data Model



Cassandra – Data Model

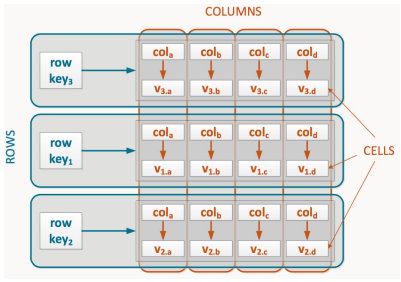
- Column families
  - Are a set of rows with a similar structure
  - Can occur in an arbitrary number per row
  - The number of column families per table is not limited
  - It is expected that only a few of them are specified at the table creation time:
    - These must be defined in advance before a cluster of servers comprising a Cassandra instance is launched

Cassandra – Data Model

- Column families
  - Consists of columns which can be added dynamically (i.e., at runtime) to column families and are not restricted in number.
  - In analogy with relational databases
    - A standard column family is as a table
    - Each key-value pair being a row

Cassandra – Data Model

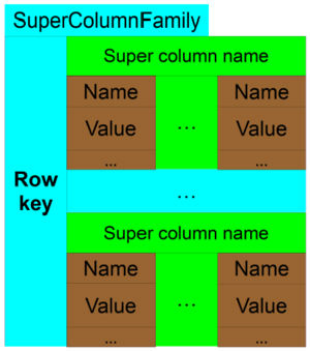
- Column Family



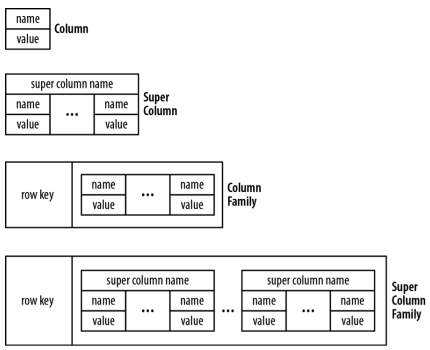
Cassandra – Data Model

- Super column families
  - When rows contain super columns, then the collection is called a super column family
  - It is a tuple (pair) that consists of a key-value pair
  - The key is mapped to a value that are column families
  - In analogy with relational databases, a super column family is something like a view on several tables
  - It can also be a map of tables

Cassandra – Data Model



Cassandra – Data Model



Cassandra – Data Model

- Clusters
  - Cassandra database is specifically designed to be distributed over several machines operating together
  - These machines appear as a single instance to the end user
  - The outermost structure in Cassandra is the cluster
  - The cluster is built on these machines
  - The cluster is sometimes called the ring
    - Cassandra assigns data to nodes in the cluster by arranging them in a ring.

Cassandra – Data Model

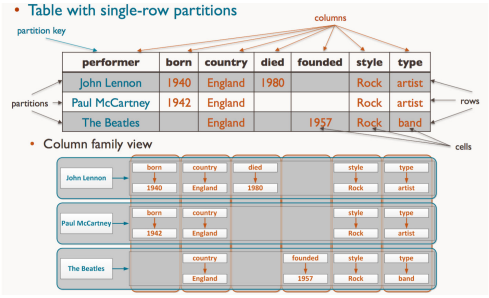
- Keyspaces
  - A cluster is a container for keyspaces.
  - A keyspace is the outermost container for data in Cassandra
  - In analogy with relational databases:
    - The keyspace corresponds closely to a relational database (schema)
    - In the same way that a database (schema) is a container for tables in the relational model, a keyspace is a container for tables in the Cassandra data model.
  - A keyspace has a name and a set of attributes that define keyspace-wide behavior.

Cassandra – Data Model

- Tables
  - A table is a container for an ordered collection of rows (records)
  - Each row (record) is itself an ordered collection of columns values
  - The ordering is determined by the columns
    - When you write data to a table in Cassandra, you specify values for one or more columns.
    - The column names are identified as keys
    - At least one of the values you specify must be a row key that serves as the unique identifier for that record.

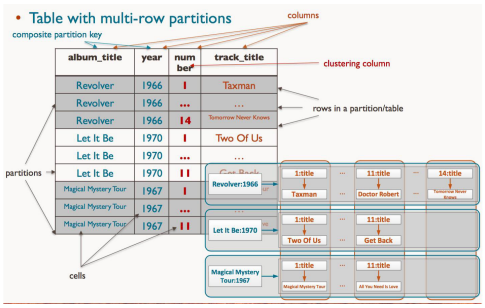
Cassandra – Data Model

- Table with single-row partitions



Cassandra – Data Model

- Table with multi-row partitions



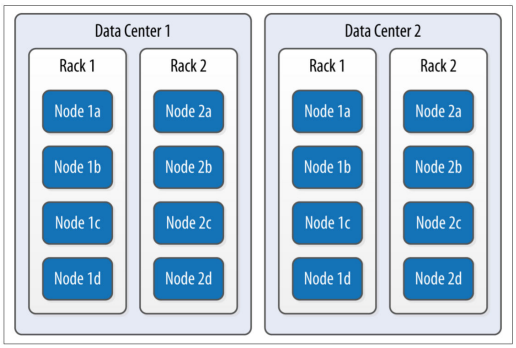
Data Centers and Racks

- Cassandra is frequently used in systems spanning physically separate locations
- Cassandra provides two levels of grouping that are used to describe the topology of a cluster:
  - Data center
  - Rack

Data Centers and Racks

- A rack
  - Is a logical set of nodes in close proximity to each other
  - Perhaps on physical machines in a single rack of equipment
- A data center
  - Is a logical set of racks
  - Is located
    - In the same building
    - At different locations
  - The racks are connected by reliable network

Data Centers and Racks



Data Centers and Racks

- Cassandra leverages the information you provide about your cluster's topology to determine
  - Where to store data
  - How to route queries efficiently
- Cassandra tries to store copies of your data in multiple data centers
  - To maximize availability
  - For partition tolerance
- Cassandra prefers to route queries to nodes in the local data center to maximize performance

Data Centers and Racks

- There are several key use cases for deploying Cassandra across multiple data centers:
  - Live backup
  - Failover
  - Load balancing
  - Geographic distribution
  - Online analysis

Data Centers and Racks - Live backups -

- Traditional database backups involve:
  - Taking periodic snapshots of the data
  - Storing them offsite in case the system fails
- In the case of failure there will be downtime as a new system is brought up and the data is restored
- This strategy also inevitably leads to data loss for the time period between the last backup and the point of failure
- Snapshot backups are still useful to protect against:
  - Data corruption
  - Accidental updates

Data Centers and Racks - Live backups -

- Cassandra's data center support can be used to provide a current backup for cases such as hardware failures
- The basic idea involves setting up a second data center that maintains a current set of replicas
- These replicas can be used to rebuild the primary cluster, should a catastrophic event cause the loss of an entire data center
- It is typically enough to maintain a smaller cluster with a replication factor of 1
- This system will never be used to accept live reads or writes
- The primary consideration in this case is storage capacity to handle the same quantity of data as the live data center

Data Centers and Racks - Failover -

- The failover scenario is very similar to the backup use case
- In this case, the backup data center is generally allocated similar resources as the primary cluster:
  - For a simple backup center, a single replica may be enough
  - For a failover data center, all nodes should be configured with the same replication factor as the primary
- With this strategy, any node can take over responsibility for the full application load in the event of a failure

Data Centers and Racks - Failover -

- When you build a failover data center, consider whether you expect your failover data center to handle a full production load
- In this case, you need to ensure that the cluster has adequate capacity to handle the entire production load
- Consider that having a hot failover data center protects you from a common single point of failure: the power supply to your hosts

Data Centers and Racks - Failover -

- Failover considerations:
  - It would be ill-advised to assume you can maintain a small failover data center, and then simply add multiple nodes in the case of failure
  - The additional overhead of bootstrapping the new nodes would reduce the capacity at a critical point when the capacity is needed most
- Bootstrapping refers to the process of loading the basic software into the memory of a computer after power-on or general reset, especially the operating system which will then take care of loading other software as needed.

Data Centers and Racks - Load balancing -

- Applications may be configured to route traffic to any node in the cluster without considering a specific data center
- This has the effect of load balancing the requests across multiple data centers
- Can be useful in cases where the data centers share a high-bandwidth connection
- The objective is to provide redundancy:
  - Each data center must be able to handle the entire application load
  - Similar to the failover scenario

Data Centers and Racks - Load balancing -

- Load balancing considerations:
  - Absolute consistency is expensive to guarantee for load balancing:
    - Consistency requires replicating the data across the entire cluster
    - For strong consistency, employing a geographic distribution model is a good strategy
  - Load balancing is appropriate for use cases where eventual consistency is acceptable:
    - The primary read case involves offline data analysis rather than real-time queries
    - Application examples: event capture, time-series data, logging, etc.

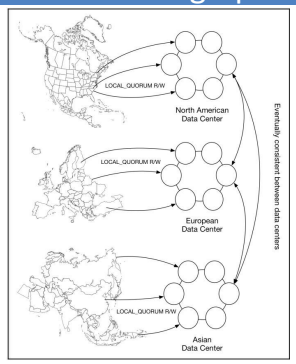
Data Centers and Racks - Geographic distribution -

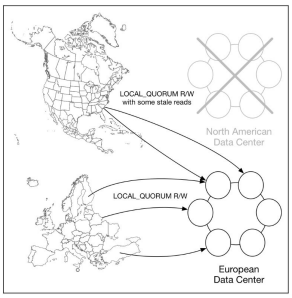
- Some application find it necessary to send requests to a data center located near the originator
- Geographic distribution is useful for systems that span the globe
- Routing all requests to a central location is impractical
- Reasons for geographic distribution
  - To reduce the latency between the originator and data centers
  - To mitigate the potential impact of natural disasters

Data Centers and Racks - Geographic distribution -

- Features of geographic distribution
  - The ability to locate data centers in strategic global locations around the world
  - Improve performance
  - Support strong consistency
- To take advantage of these features:
  - Enable the use of the LOCAL_QUORUM consistency level to remove the performance penalty introduced by waiting for a remote data center to acknowledge the write

Data Centers and Racks - Geographic distribution -





Data Centers and Racks - Data analysis -

- Data analysis (Online Analytical Processing – OLAP) involves:
  - Taking normalized data from the transactional relational database
  - Moving it into a denormalized form for faster analysis
- This process involves significant Extract Transform and Load (ETL) overhead
- ETL inherently results in a delay in analyzing the data

Data Centers and Racks - Data analysis -

- Cassandra offers for data analysis:
  - A denormalized data model as default
  - Support of multiple data centers
  - Support replication for load balancing
  - Robust integrations with the Hadoop
  - Robust integrations with the Spark

Gossip and Failure Detection

- Cassandra uses a gossip protocol to support decentralization and partition tolerance
- The gossip protocol allows each node to keep track of state information about the other nodes in the cluster
- The gossiper runs every second on a timer
- The gossip protocols take their name from the concept of human gossip
  - A form of communication in which peers can choose with whom they want to exchange information

Gossip and Failure Detection

- The gossip protocols
  - Generally, assume a faulty network
  - Are commonly employed in very large, decentralized network systems
  - Are often used as an automatic mechanism for replication in distributed databases.
  - Cassandra is primarily implemented by the org.apache.cassandra.gms.Gossiper class, which is responsible for managing gossip for the local node

Gossip and Failure Detection

- Cassandra gossip is used for failure detection
- The Gossiper class maintains a list of nodes that are alive and dead
- When a server node is started:
  - It registers itself with the gossiper
  - It will receive from the gossiper the endpoint state information

Gossip and Failure Detection

- The gossiper works as follows:

1. Once per second, the gossiper:
   I. Will choose a random node in the cluster (friend)
   II. Will initialize a gossip session with the chosen friend

2. Each round of gossip requires three messages:
   I. The gossip initiator sends its chosen friend a GossipDigestSynMessage
   II. When the friend receives this message, it returns a GossipDigestAckMessage
   III. When the initiator receives the ack message from the friend, it sends the friend a GossipDigestAck2Message to complete the round of gossip.

Gossip and Failure Detection

- When the gossiper determines that another endpoint is dead:
  - "Convicts" that endpoint
  - Marks it as dead in its local list
  - Logs the modification
- Cassandra has robust support for failure detection
- Failure detections uses the algorithm for distributed computing called Phi Accrual Failure Detection.

Gossip and Failure Detection

- Phi Accrual Failure Detectors
  - Accrual Failure Detectors output a value associated with each process (or node)
  - This value is called Phi
  - The value is output in a manner that is designed from the ground up to be adaptive in the face of volatile network conditions
  - Phi is not a binary condition that simply checks whether a server is up or down
  - The Phi convict threshold in the configuration adjusts the sensitivity of the failure detector

Gossip and Failure Detection

- Phi Accrual Failure Detectors
  - Lower values increase the sensitivity to failure
  - Higher values decrease the sensitivity to failure
  - This process is not done in a linear fashion.
  - The Phi value refers to a level of suspicion that a server might be down.
  - Applications such as Cassandra that employ an AFD can specify variable conditions for the Phi value they emit.
  - Cassandra can generally detect a failed node in about 10 seconds using this mechanism

Gossip and Failure Detection

- Cassandra's accrual failure detection is based on two primary ideas:

1. The first general idea is that failure detection should be flexible
   - This is achieved by decoupling it from the application being monitored

2. The second and more novel idea challenges the notion of traditional failure detectors
   - Traditional failure detectors
     - Are implemented by simple "heartbeats"
     - Decide whether a node is dead or not dead based on whether a heartbeat is received or not
   - Accrual failure detection considers this approach as naive and finds a place in between the extremes of dead and alive - a suspicion level

Gossip and Failure Detection

- Cassandra's accrual failure
  - The failure monitoring system outputs a continuous level of "suspicion" regarding how confident it is that a node has failed.
  - This is desirable because it can consider fluctuations in the network environment.
  - For example, just because one connection gets caught up doesn't necessarily mean that the whole node is dead.
  - So, suspicion offers a more fluid and proactive indication of the weaker or stronger possibility of failure based on interpretation (the sampling of heartbeats), as opposed to a simple binary assessment

Rings and Tokens

- Cassandra represents the data managed by a cluster as a ring
- Each node in the ring is assigned one or more ranges of data described by a tokens
- The token
  - Is **the hashed value of the primary key (row/partition key)**
  - Determines the nodes position in the ring
  - Is a 64-bit integer ID used to identify each partition
- Because the token is a 64-bit integer ID, it gives a possible range for tokens in the range $[-2^{63}, 2^{63} - 1]$

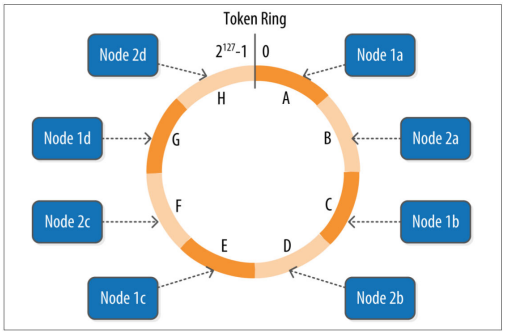
Rings and Tokens

- A node claims ownership of the range of values
  - Less than or equal to each token
  - Greater than the token of the previous node
- The node with lowest token owns the range
  - Less than or equal to its token
  - Greater than the highest token
- This range is also known as the "wrapping range“
- In this way, the tokens specify a complete ring

Rings and Tokens

- The ring is structured such that consecutive token ranges are spread across nodes in different racks
- Data is assigned to nodes by using a hash function to calculate a token for the partition key
- This partition key token is compared to the token values for the various nodes to identify the range, and therefore the node, that owns the data

Rings and Tokens



Virtual Nodes

- Cassandra's 1.2 release introduced the concept of virtual nodes, also called **vnodes** for short
- Instead of assigning a single token to a node, the token range is broken up into multiple smaller ranges
- Each physical node is then assigned multiple tokens
- By default, each node will be assigned 256 of these tokens, meaning that it contains 256 virtual nodes

Virtual Nodes

- Virtual nodes have been enabled by default since 2.0.
- Vnodes make it easier to maintain a cluster containing heterogeneous machines
- For nodes in your cluster that have more computing resources available to them, you can increase the number of **vnodes** by setting the **num_tokens** property in the **cassandra.yaml** file
- Conversely, you might **set num_tokens** lower to decrease the number of **vnodes** for less capable machines.

Virtual Nodes

- Cassandra automatically handles the calculation of token ranges for each **vnode (can in consequence for each node)** in the cluster in proportion to their **num_tokens** value
- A further advantage of virtual nodes is that they speed up some of the more heavy-weight Cassandra operations such as
  - Bootstrapping a new node
  - Decommissioning a node
  - Repairing a node
- This is because the load associated with operations on multiple smaller ranges is spread more evenly across the nodes in the cluster.

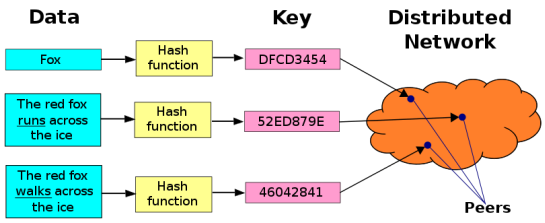
Partitioners

- A **partitioner** determines how data is distributed across the nodes in the cluster
- Cassandra stores data in wide rows, or "partitions“
- Each row has a **partition key** that is used to identify the partition
- A **partitioner** is a hash function for computing the token of a partition key
- Each row of data is distributed within the ring according to the value of the **partition key token**

Partitioners

- Cassandra provides several different **partitioners** that use a distributed hash table (DHT)
- A DHT is a class of a decentralized distributed system that provides a lookup service similar to a hash table
- For hashing it uses a Java implementation of the murmur algorithm that generates 64-bit hashes
- MurmurHash is a non-cryptographic hash function suitable for general hash-based lookup
- Because of Cassandra's generally pluggable design, you can also create your own **partitioner**

Partitioners



Replication Strategies

- A node serves as a replica for different ranges of data
- If one node goes down, other replicas can respond to queries for that range of data
- Cassandra replicates data across nodes in a manner transparent to the user
- The replication factor is the number of nodes in your cluster that will receive copies (replicas) of the same data

Replication Strategies

- If the replication factor is 3, then three nodes in the ring will have copies of each row
- The first replica will always be the node that claims the range in which the token falls
- The remainder of the replicas are placed according to the replication strategy (or replica placement strategy)

Replication Strategies

- Cassandra provides two primary replication strategies (we will discuss others in the next course):
  - The SimpleStrategy
    - Places replicas at consecutive nodes around the ring
    - It starts with the node indicated by the partitioner
  - The NetworkTopologyStrategy
    - That allows the administrator to specify a different replication factor for each data center
    - Within a data center, it allocates replicas to different racks in order to maximize availability

Consistency Levels

- Cassandra provides tuneable consistency levels that allow you to make trade-offs between consistency, availability, and partition tolerance at a fine-grained level
- Cassandra lets the client specify a consistency level on each read or write query that indicates how much consistency you require
- A higher consistency level means that more nodes need to respond to a read or write query, ensuring that the values present on each replica are the same

Consistency Levels

- For read queries, the consistency level specifies how many replica nodes must respond to a read request before returning the data
- For write operations, the consistency level specifies how many replica nodes must respond for the write to be reported as successful to the client
- Because Cassandra is eventually consistent, updates to other replica nodes may continue in the background

Consistency Levels

- The available consistency levels include ANY, ONE, TWO, and THREE, each of which specify an absolute number of replica nodes that must respond to a request
- The QUORUM consistency level requires a response from a majority of the replica nodes (N is the replication factor): N/2 + 1
- The ALL consistency level requires a response from all of the replicas
- For both reads and writes, the consistency levels of
  - ANY, ONE, TWO , and THREE are considered weak
  - QUORUM and ALL are considered strong

Consistency Levels

- Consistency is tuneable in Cassandra because clients can specify the desired consistency level on both reads and writes
- There is an equation that is popularly used to represent the way to achieve strong consistency in Cassandra: R + W > N = strong consistency
- In this equation
  - R the read replica count
  - W the write replica count
  - N the replication factor
- All client reads will see the most recent write in this scenario, and you will have strong consistency

Queries and Coordinator Nodes

- A client may connect to any node in the cluster to initiate a read or write query
- This node is known as the coordinator node
- The coordinator identifies which nodes are replicas for the data that is being written or read and forwards the queries to them

Queries and Coordinator Nodes

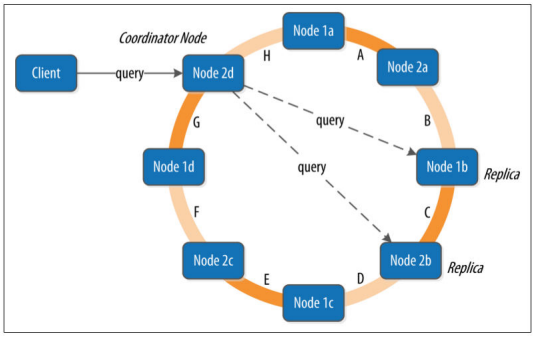
- Write:

1. The coordinator node contacts all replicas, as determined by the consistency level and replication factor
2. Considers the write successful when a number of replicas commensurate with the consistency level acknowledge the write

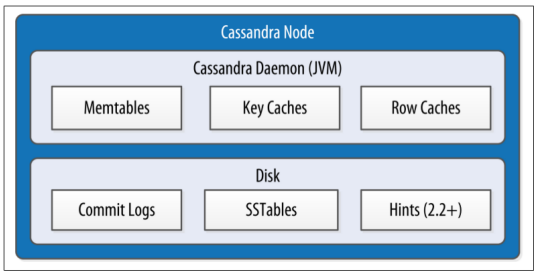
- Read:

1. The coordinator contacts enough replicas to ensure the required consistency level is met
2. Returns the data to the client

Queries and Coordinator Nodes



Commit Logs, Memtables and SSTables



Commit Logs, Memtables and SSTables

- Cassandra stores data both in memory and on disk to provide both high performance and durability
- To support the writing and reading of data from tables, Cassandra uses
  1. Commit logs
  2. Memtables
  3. SSTables
  4. Key Caches
  5. Row Caches
  6. Hints

Commit Logs, Memtables and SSTables

- Commit Logs
  - When a write operation is performed, it is immediately written to a commit log
  - The commit log is a crash-recovery mechanism that supports Cassandra's durability goals
  - A write will not count as successful until it's written to the commit log
  - This ensures that if a write operation does not make it to the in-memory store (memtable) on on disk store (SSTable), it will still be possible to recover the data
    - The commit log can ensure that data is not lost if the database is shut down or it crashes unexpectedly
    - That's because the next time you start the node, the commit log gets replayed
    - In fact, that's the only time the commit log is read – clients never read from it

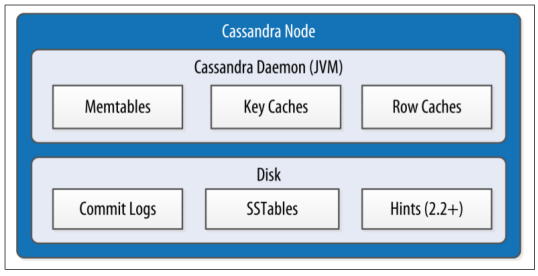
Commit Logs, Memtables and SSTables

- Commit Logs
  - Each commit log maintains an internal bit flag to indicate whether it needs flushing
  - When a write operation is first received, it is written to the commit log and its bit flag is set to 1
  - There is only one bit flag per table, because only one commit log is ever being written to across the entire server
  - All writes to all tables will go into the same commit log, so the bit flag indicates whether a particular commit log contains anything that hasn't been flushed for a particular table
  - After it's written to the commit log, the value is written to a memory-resident data structure called the memtable

Commit Logs, Memtables and SSTables

- Commit Logs
  - Once the memtable has been properly flushed to disk, the corresponding commit log's bit flag is set to 0, indicating that the commit log no longer has to maintain that data for durability purposes
  - Like regular logfiles, commit logs have a configurable rollover threshold, and once this file size threshold is reached, the log will roll over, carrying with it any extant dirty bit flags

Commit Logs, Memtables and SSTables



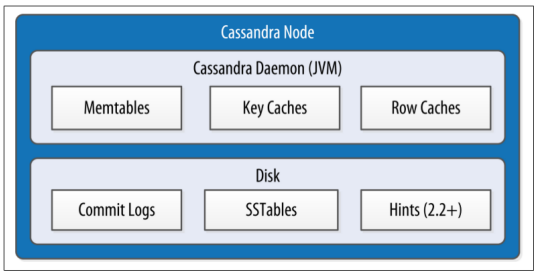
Commit Logs, Memtables and SSTables

- Memtables
  - Each memtable contains data for a specific table
  - In early implementations of Cassandra, memtables were stored on the JVM heap, but now the majority of memtable data was moved to native memory
  - This makes Cassandra less susceptible to fluctuations in performance due to Java garbage collection

Commit Logs, Memtables and SSTables

- Memtables
  - When the number of objects stored in the memtable reaches a threshold, the contents of the memtable are flushed to disk in a file called an SSTable.
  - A new memtable is then created.
  - This flushing is a non-blocking operation; multiple memtables may exist for a single table, one current and the rest waiting to be flushed.
  - They typically should not have to wait very long, as the node should flush them very quickly unless it is overloaded

Commit Logs, Memtables and SSTables



Commit Logs, Memtables and SSTables

- SSTables
  - Once a memtable is flushed to disk as an SSTable, it is immutable and cannot be changed by the application.
  - Even though SSTables are compacted, this compaction changes only their on-disk representation.
  - It essentially performs the "merge" step of a mergesort into new files and removes the old files on success.
  - Cassandra supports the compression of SSTables in order to maximize use of the available storage.
  - This compression is configurable per table.
  - Each SSTable also has an associated Bloom filter, which is used as an additional performance enhancer

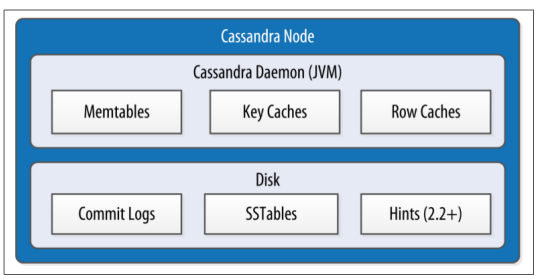
Commit Logs, Memtables and SSTables

- SSTables
  - All writes are sequential, which is the primary reason that writes perform so well in Cassandra
  - No reads or seeks of any kind are required for writing a value to Cassandra because all writes are append-only operations
  - This makes one key limitation on performance the speed of your disk
  - Compaction is intended to minimize the reorganization of data: uses sequential I/O

Commit Logs, Memtables and SSTables

- SSTables
  - The write operation is just an immediate append
  - Compaction helps to organize for better future read performance
  - If Cassandra naively inserted values where they ultimately belonged, writing clients would pay for seeks up front
  - On reads, Cassandra will read both SSTables and memtables to find data values, as the memtable may contain values that have not yet been flushed to disk.

Caching



Caching

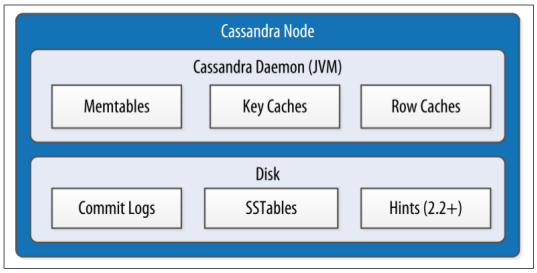
- Cassandra provides three forms of caching:

1. Key caches
   - Stores a map of partition keys to row index entries
   - Facilitating faster read access into SSTables stored on disk
   - The key cache is stored on the JVM heap
2. Row caches
   - Uses entire rows
   - Can greatly speed up read access for frequently accessed rows, at the cost of more memory usage
   - The row cache is stored in off-heap memory
3. The counter cache
   - Developed to improve counter performance by reducing lock contention for the most frequently accessed counters

Caching

- By default, key and counter caching are enabled
- Row caching is disabled by default because it requires more memory
- Cassandra saves its caches to disk periodically in order to warm them up more quickly on a node restart

Hinted Handoff



Hinted Handoff

- A write request is sent to Cassandra, but a replica node where the write properly belongs is not available (due to network partition, hardware failure, or some other reason)
- In this case, the coordinator will create a hint
- This is used by Cassandra to ensure general availability in such cases
- Cassandra holds a separate hint for each partition that is to be written
- This allows Cassandra to
  - Be always available for writes
  - Enables a cluster to sustain the same write load even when some of the nodes are down
- It also reduces the time that a failed node will be inconsistent after it does come back online

Installation

- To install Cassandra on a single node:
  - Download the archive

$ wget https://dlcdn.apache.org/cassandra/5.0.2/apache-cassandra-5.0.2-bin.tar.gz

  - Unzip the archive

$ tar -zxvf apache-cassandra-5.0.2-bin.tar.gz

  - Run Cassandra

$ cd ./apache-cassandra/bin
$ ./cassandra

Cluster Installation

- Installing Cassandra on a cluster (follow all the steps on each node)
  - Download the archive
  - Unzip the archive
  - Edit the cassandra.yaml ($CASSANDRA_HOME/conf/cassandra.yaml)
  - Start Cassandra on each node

Cluster Installation

- Edit the cassandra.yaml
  - cluster_name:
    - This is the name of your cluster.
  - - seeds:
    - This is a comma-delimited list of the IP address of each node in the cluster.
  - listen_address:
    - This is IP address that other nodes in the cluster will use to connect to this one.
    - It defaults to localhost and needs changed to the IP address of the node.

Cluster Installation

- Edit the cassandra.yaml
  - rpc_address:
    - This is the IP address for remote procedure calls.
    - It defaults to localhost.
    - If the server's hostname is properly configured, leave this as is.
    - Otherwise, change to server's IP address or the loopback address (127.0.0.1).
  - endpoint_snitch:
    - Name of the snitch, which is what tells Cassandra about what its network looks like.
    - This defaults to SimpleSnitch, which is used for networks in one datacenter.

Cluster Installation

- Edit the cassandra.yaml
  - auto_bootstrap:
    - This directive is not in the configuration file, so it has to be added and set to false.
    - This makes new nodes automatically use the right data.
    - It is optional if you're adding nodes to an existing cluster, but required when you're initializing a fresh cluster, that is, one with no data.

Cluster Installation

- Out of the box, Cassandra provides for endpoint_snitch:
  - SimpleSnitch:
    - Treats Strategy order as proximity.
    - This can improve cache locality when disabling read repair.
    - Only appropriate for single-datacenter deployments.
  - GossipingPropertyFileSnitch
    - This should be your go-to snitch for production use.
    - The rack and datacenter for the local node are defined in cassandra-rackdc.properties and propagated to other nodes via gossip.
    - If cassandra-topology.properties exists, it is used as a fallback, allowing migration from the PropertyFileSnitch.

Cluster Installation

- Out of the box, Cassandra provides for endpoint_snitch:
  - PropertyFileSnitch:
    - Proximity is determined by rack and data center, which are explicitly configured in cassandra-topology.properties.
  - Ec2Snitch:
    - Appropriate for EC2 deployments in a single Region.
    - Loads Region and Availability Zone information from the EC2 API.
    - The Region is treated as the datacenter, and the Availability Zone as the rack.
    - Only private IPs are used, so this will not work across multiple Regions.

Cluster Installation

- Out of the box, Cassandra provides for endpoint_snitch:
  - Ec2MultiRegionSnitch:
    - Uses public IPs as broadcast_address to allow cross-region connectivity.
    - Thus, you should set seed addresses to the public IP as well.
    - You will need to open the storage_port or ssl_storage_port on the public IP firewall.
    - For intra-Region traffic, Cassandra will switch to the private IP after establishing a connection.

Cluster Installation

- Out of the box, Cassandra provides for endpoint_snitch:
  - RackInferringSnitch:
    - Proximity is determined by rack and data center, which are assumed to correspond to the 3rd and 2nd octet of each node's IP address, respectively.
    - Unless this happens to match your deployment conventions, this is best used as an example of writing a custom Snitch class and is provided in that spirit.

Cluster Installation

```
# The name of the cluster. This is mainly used to prevent machines in
# one logical cluster from joining another.
cluster_name: 'DBCassandraCluster'

...

# any class that implements the SeedProvider interface and has a
# constructor that takes a Map<String, String> of parameters will do.
seed_provider:
    # Addresses of hosts that are deemed contact points.
    # Cassandra nodes use this list of hosts to find each other and learn
    # the topology of the ring.  You must change this if you are running
    # multiple nodes!
    - class_name: org.apache.cassandra.locator.SimpleSeedProvider
      parameters:
          # seeds is actually a comma-delimited list of addresses.
          # Ex: "<ip1>,<ip2>,<ip3>"
          - seeds: "172.19.3.31,172.19.3.32,172.193.33"
```


Cluster Installation

```
# Address or interface to bind to and tell other Cassandra nodes to connect to.
# You _must_ change this if you want multiple nodes to be able to communicate!
#
# Set listen_address OR listen_interface, not both.
#
# Leaving it blank leaves it up to InetAddress.getLocalHost(). This
# will always do the Right Thing _if_ the node is properly configured
# (hostname, name resolution, etc), and the Right Thing is to use the
# address associated with the hostname (it might not be).
#
# Setting listen_address to 0.0.0.0 is always wrong.
#
listen_address: 172.19.3.31

...

# The address or interface to bind the Thrift RPC service and native transport
# server to.
#
# Set rpc_address OR rpc_interface, not both.
#
# Leaving rpc_address blank has the same effect as on listen_address
# (i.e. it will be based on the configured hostname of the node).
#
# Note that unlike listen_address, you can specify 0.0.0.0, but you must also
# set broadcast_rpc_address to a value other than 0.0.0.0.
#
# For security reasons, you should not expose this port to the internet.  Firewall it if needed.
rpc_address: 172.19.3.31
```


Cluster Installation

```
# You can use a custom Snitch by setting this to the full class name
# of the snitch, which will be assumed to be on your classpath.
endpoint_snitch: GossipingPropertyFileSnitch


auto_bootstrap: false
```


Cluster Installation

- To verify the status of you cluster use nodetool utility.

$ nodetool status

```
Datacenter: dc1
===============
Status=Up/Down
|/ State=Normal/Leaving/Joining/Moving
--  Address      Load       Tokens       Owns (effective)  Host ID                               Rack
UN  172.19.3.33  321.07 KiB  256          100.0%            26e4ec87-b562-448e-af70-7e62dba50abe  rack1
UN  172.19.3.32  348.49 KiB  256          100.0%            90a71c7a-15ac-41b8-8437-78984d89b7e4  rack1
UN  172.19.3.31  322.63 KiB  256          100.0%            89cec698-d686-4d97-9024-088babd9ebc3  rack1
```

CQL

- Cassandra Query Language (CQL)
- The syntax of CQL is similar in many ways to SQL, but with some important differences.
- To start the CLI use cqlsh ($CASSANDRA_HOME/bin/cqlsh)
- The default port for cqlsh is 9042
- If the server is listening on localhost then issue the following command:

```
$ cqlsh
```

- If the server is listening on a different IP/PORT (used by one of the nodes in the cluster), use

```
$ cqlsh IP PORT
```


CQL

- To see all the commands use HELP
- See the cluster details

```
cqlsh> DESCRIBE CLUSTER;
```

- To see the keyspaces available:

```
cqlsh> DESCRIBE KEYSPACES;
```

- Initially this list will consist of several system keyspaces.
- Once you have created your own keyspaces, they will be shown as well.
- The system keyspaces are managed internally by Cassandra.
- To see the client, server, and protocol versions:

```
cqlsh> SHOW VERSION;
```


CQL
Creating a Keyspace

- When creating a keyspace, there are different strategies to choose from:

1. SimpleStrategy
   - Places replicas at consecutive nodes around the ring, starting with the node indicated by the partitioner.
   - The replication factor must be specified
2. NetworkTopologyStrategy
   - Allows you to specify a different replication factor for each data center
3. OldNetworkTopologyStrategy - deprecated staring with v4
   - Is similar to the NetworkTopologyStrategy in that it places replicas in multiple data centers, but its algorithm is less sophisticated.
   - This is depricate, it was keept just for backwards compatibility.

CQL

- The following command creats a Keyspace

```
cqlsh> CREATE KEYSPACE faculty WITH replication = {'class': 'SimpleStrategy', 'replication_factor': 3 };
```

- To bypass writing to the commit log for the keyspace then you can use durable_writes property.

```
cqlsh> CREATE KEYSPACE faculty WITH replication = {'class': 'SimpleStrategy', 'replication_factor': 3 } AND durable_writes = true;
```

- To delete a Keyspace use the DROP command

```
cqlsh> DROP KEYSPACE faculty;
```

- To use a keyspace and start creating tables:

```
cqlsh> USE faculty
```


CQL

- To creating a table, use the CREATE TABLE command

```
cqlsh:faculty> CREATE TABLE TEACHERS(
                    id int PRIMARY KEY,
                    firstname VARCHAR,
                    lastname VARCHAR,
                    age INT );
```

- Tables must have a primary key upon creation
- To modify a table, use ALTER TABLE command
- To drop a table, use DROP TABLE command

```
cqlsh:faculty> DROP TABLE teachers;
```


CQL

- Adding data to a table use the INSERT command
- The columns need to be specified

```
cqlsh:faculty> INSERT INTO teachers(id, firstname, lastname, age) VALUES(1, 'Ciprian-Octavian', 'Truică', 32);

cqlsh:faculty> INSERT INTO teachers(id, firstname, lastname, age) VALUES(2, 'Florin', 'Rădulescu', 58);

cqlsh:faculty> INSERT INTO teachers(id, firstname, lastname, age) VALUES(3, 'Alexandru', 'Boicea', 59);

cqlsh:faculty> INSERT INTO teachers(id, firstname, lastname, age) VALUES(4, 'Sorin', 'Ciolofan', 37);

cqlsh:faculty> INSERT INTO teachers(id, firstname, lastname, age) VALUES(5, 'Dan', 'Schrager', 59);
```


CQL

- Selection and projection is done using the SELECT command

```
cqlsh:faculty> SELECT * FROM teachers;

cqlsh:faculty> SELECT firstname, lastname FROM teachers
WHERE id IN (1, 3);
```

- When filtering we need to specify that we are filtering the data using ALLOW FILTERING property

```
cqlsh:faculty> SELECT firstname, lastname FROM teachers
WHERE age <= 40 ALLOW FILTERING;
```


CQL

- Some limitations on SELECT command:
  - There are no nested queries
  - Some operators don't work, e.g., between ... and ... ,
  - Some operators work only on primary keys, e.g., IN.
  - There are no JOIN operations

CQL

- Update a value

```
cqlsh:faculty> UPDATE teachers SET lastname = 'Truică' WHERE id = 1;

cqlsh:faculty> UPDATE teachers SET lastname = 'Truică', firstname = 'Ciprian'
WHERE id = 1;
```

- Delete a value

```
cqlsh:faculty> DELETE FROM teachers WHERE id = 5;
```

- Delete all values

```
cqlsh:faculty> TRUNCATE teachers;
```


CQL Data Types

- CQL Data Types
  - Numeric Data Types
  - Textual Data Types
  - Time and Identity Data Types
  - Other Simple Data Types
  - Collections
  - User-Defined Types

CQL Data Types
Numeric Data Types

- Integer and floating-point numbers.
- These types are similar to standard types in Java:
  - int - A 32-bit signed integer (as in Java)
  - bigint - A 64-bit signed long integer (equivalent to a Java long)
  - smallint - A 16-bit signed integer (equivalent to a Java short)
  - tinyint - An 8-bit signed integer (as in Java)
  - varint - A variable precision signed integer (equivalent to java.math.BigInteger)
  - float - A 32-bit IEEE-754 floating point (as in Java)
  - double - A 64-bit IEEE-754 floating point (as in Java)
  - decimal - A variable precision decimal (equivalent to java.math.BigDecimal)

CQL Data Types
Textual Data Types

- CQL provides two data types for representing text:
  - text, varchar - Synonyms for a UTF-8 character string
  - ascii - An ASCII character string
- The ascii type is most useful if you are dealing with legacy data that is in ASCII format.

CQL Data Types
Time and Identity Data Types

- The identity of data elements such as rows and partitions is important in any data model in order to be able to access the data.
- Cassandra provides several types which prove quite useful in defining unique partition keys.
- Time Data Types:
  - timestamp
  - date and time
  - uuid
  - timeuuid

CQL Data Types
Time and Identity Data Types

- timestamp
  - Each column has a timestamp indicating when it was last modified
  - A timestamp can also be used as the value of a column itself
  - The time can be encoded as a 64-bit signed integer
  - It is typically much more useful to input a timestamp using one of several supported ISO 8601 date formats
  - The best practice is to always provide time zones rather than relying on the operating system time zone configuration

CQL Data Types
Time and Identity Data Types

- date, time
  - Releases through Cassandra 2.1 only had the timestamp type to represent times, which included both a date and a time of day.
  - The 2.2 release introduced date and time types that allowed these to be represented independently, that is:
    - A date without a time
    - A time of day without reference to a specific date.

CQL Data Types
Time and Identity Data Types

- uuid
  - Universally unique identifier (UUID) is a 128-bit value in which the bits conform to one of several types, of which the most commonly used are known as Type 1 and Type 4.
  - The CQL uuid type is a Type 4 UUID, which is based entirely on random numbers.
  - UUIDs are typically represented as dash-separated sequences of hex digits.
  - The uuid type is often used as a surrogate key, either by itself or in combination with other values.
  - You can obtain a Type 4 UUID value via the uuid() function and use this value in an INSERT or UPDATE.

CQL Data Types
Time and Identity Data Types

- timeuuid
  - This is a Type 1 UUID, which is based on the following 3 components:
    - The MAC address of the computer
    - The system time
    - A sequence number used to prevent duplicates
  - This type is frequently used as a conflict-free timestamp
  - cqlsh provides several convenience functions for interacting with the timeuuid type:
    - now()
    - dateOf()
    - unixTimestampOf().

CQL Data Types
Other Simple Data Types

- boolean
  - This is a simple true/false value.
  - The cqlsh is case insensitive in accepting these values but outputs True or False
- inet
  - This type represents IPv4 or IPv6 Internet addresses.
  - cqlsh accepts any legal format for defining IPv4 addresses, including:
    - Dotted or non-dotted representations containing:
      - Decimal values.
      - Octal values.
      - Hexadecimal values.
  - IPv6 addresses are represented as eight groups of four hexadecimal digits, separated by colons

CQL Data Types
Other Simple Data Types

- counter
  - The counter data type provides 64-bit signed integer
  - Its value cannot be set directly, but only incremented or decremented.
  - Cassandra is one of the few databases that provides race-free increments across data centers
  - Counters are frequently used for tracking statistics such as numbers of page views, tweets, log messages, and so on.
  - The counter type has some special restrictions: It cannot be used as part of a primary key.

CQL Data Types
Other Simple Data Types

- counter
  - The increment and decrement operators are not idempotent.
  - There is no operation to reset a counter directly, but you can approximate a reset by reading the counter value and decrementing by that value.
  - Unfortunately, this is not guaranteed to work perfectly, as the counter may have been changed elsewhere in between reading and writing.

CQL Data Types

- CQL Collections
  - Set
  - List
  - Map

CQL Data Types
Set

- The set data type stores a collection of elements
- The elements are unordered, but cqlsh returns the elements in sorted order
- Sets can contain:
  - Simple types
  - User-defined types
  - Other collections
- One advantage of using set is the ability to insert additional items without having to read the contents first

CQL Data Types
Set

```
cqlsh:faculty> ALTER TABLE teachers ADD offices SET<TEXT>;

cqlsh:faculty> UPDATE teachers SET offices= {'EF208', 'PR305', 'EG105'} WHERE id=1;

cqlsh:faculty> UPDATE teachers SET offices= {'Vice Dean Office'} WHERE id=2;

cqlsh:faculty> UPDATE teachers SET offices= {'EF309'} WHERE id=3;

cqlsh:faculty> UPDATE teachers SET offices= {'EF209'} WHERE id=4;

cqlsh:faculty> UPDATE teachers SET offices= {'PR304'} WHERE id=5;
```


CQL Data Types
Set operations

- Adding a new element using +

```
cqlsh:faculty> UPDATE teachers SET offices= offices + {'EF309'} WHERE id =2;
```

- Removing an element using -

```
cqlsh:faculty> UPDATE teachers SET offices= offices - {'EG105'} WHERE id =1;
```

- Empty the set use {}

```
cqlsh:faculty> UPDATE teachers SET offices= { } WHERE id =4;
```

- Select

```
cqlsh:faculty> SELECT firstname, lastname FROM teachers WHERE offices
CONTAINS 'EF309' ALLOW FILTERING;
```

CQL Data Types
List

- The list data type contains an unordered list of elements.
- By default, the values are stored in order of insertion
- Uses the same operations as sets to add and remove an element
- The empty a list is []
- Indexes can be used to access elements (in the WHERE clause of SELECT and SET clause of UPDATE).
- An element can be deleted using the index.

CQL Data Types
List

```
cqlsh:faculty> ALTER TABLE teachers ADD telephone list<text>;

cqlsh:faculty> UPDATE teachers SET telephone = ['+40 21 323.222.21', '+40
721.222.111'] WHERE id = 1;

cqlsh:faculty> UPDATE teachers SET telephone = ['+40 21 323.222.22', '+40
721.222.112'] WHERE id = 2;

cqlsh:faculty> UPDATE teachers SET telephone = ['+40 21 323.222.23', '+40
721.222.113'] WHERE id = 3;

cqlsh:faculty> UPDATE teachers SET telephone = ['+40 21 323.222.23', '+40
721.222.114'] WHERE id = 4;

cqlsh:faculty> UPDATE teachers SET telephone = ['+40 21 323.222.25', '+40
721.222.115'] WHERE id = 5;
```


CQL Data Types
List

```
cqlsh:faculty> UPDATE teachers SET telephone = telephone - ['+40 721.222.111']
WHERE id = 1;

cqlsh:faculty> UPDATE teachers SET telephone = telephone + ['+40 21 323.222.30']
WHERE id = 3;

cqlsh:faculty> UPDATE teachers SET telephone = [ ] WHERE id = 5;

cqlsh:faculty> UPDATE teachers SET telephone[0] = '+40 21 323.222.90' WHERE id = 2;

cqlsh:faculty> DELETE telephone[2] FROM teachers  WHERE id = 3;

cqlsh:faculty> SELECT firstname, lastname FROM teachers WHERE telephone
CONTAINS '+40 21 323.222.23' ALLOW FILTERING;
```


CQL Data Types
Map

- The map data type contains a collection of key/value pairs
- The keys and the values can be of any type except counter
- Collection types are very useful in cases where we need to store a variable number of elements within a single column

CQL Data Types
Map

```
cqlsh:faculty> ALTER TABLE teachers ADD title map<text,text>;

cqlsh:faculty> UPDATE teachers SET title = {'academic': 'teaching assistant',
'profession': 'engineer'} WHERE id = 1;

cqlsh:faculty> UPDATE teachers SET title = {'academic': 'professor',
'profession': 'engineer'} WHERE id = 2;

cqlsh:faculty> UPDATE teachers SET title = {'academic': 'associate professor',
'profession': 'engineer'} WHERE id = 3;

cqlsh:faculty> UPDATE teachers SET title = {'academic': 'lecturer', 'profession':
'engineer'} WHERE id = 4;
```


CQL Data Types
Map

```
cqlsh:faculty> UPDATE teachers SET title = {'academic': 'lecturer',
'profession': 'engineer'} WHERE id = 5;

cqlsh:faculty> SELECT firstname, lastname FROM teachers WHERE
title['academic'] = 'teaching assistant' ALLOW FILTERING;

cqlsh:faculty> SELECT firstname, lastname FROM teachers WHERE title
contains 'lecturer' ALLOW FILTERING;

cqlsh:faculty> SELECT firstname, lastname FROM teachers WHERE title
CONTAINS KEY 'academic' ALLOW FILTERING;
```


CQL Data Types
User Defined Types (UDT)

- Cassandra supports UDTs
- They can be created using CREATE TYPE command
- They can be used on their own or using a collection, where they need to be frozen

```
cqlsh:faculty> CREATE TYPE course(
                    name VARCHAR,
                    year INT
               );
```


CQL Data Types
UDT

```
cqlsh:faculty> ALTER TABLE teachers ADD courses list<frozen<course>>;

cqlsh:faculty> UPDATE teachers SET courses = [{name: 'BD1', year: 3}, {name: 'BD2',
year: 4}, {name: 'DM', year: 5}] WHERE id = 1;

cqlsh:faculty> UPDATE teachers SET courses = [{name: 'BD1', year: 3}, {name: 'BD2',
year: 4}, {name: 'UBD', year: 4}, {name: 'DMDW', year: 5}] WHERE id = 2;

cqlsh:faculty> UPDATE teachers SET courses = [{name: 'BD1', year: 3}, {name: 'BD2',
year: 4}, {name: 'SCAD', year: 4}, {name: 'ICASE', year: 5}] WHERE id = 3;

cqlsh:faculty> UPDATE teachers SET courses = [{name: 'PC', year: 1}, {name: 'ABD',
year: 5}] WHERE id = 4;

cqlsh:faculty> UPDATE teachers SET courses = [{name: 'PAJ', year: 5}, {name: 'MN',
year: 1}] WHERE id = 5;
```

CQL Secondary Indexes

- To create a secondary index use the CREATE INDEX command
- It's also possible to create indexes that are based on values in collections.
- For optimal read performance: denormalized table designs or materialized views are generally preferred to using secondary indexes.
- Note that for maps in particular, we have the option of indexing either the keys (via the syntax KEYS(addresses)) or the values (which is the default), or both (in Cassandra 2.2 or later).

CQL Secondary Indexes

```
cqlsh:faculty> CREATE INDEX idx_offices ON teachers(offices);

cqlsh:faculty> SELECT firstname, lastname FROM teachers WHERE offices
CONTAINS 'EF309';

cqlsh:faculty> DROP INDEX idx_offices;


cqlsh:faculty> CREATE INDEX idx_telephone ON teachers(telephone);

cqlsh:faculty> SELECT firstname, lastname FROM teachers WHERE
telephone CONTAINS '+40 21 323.222.23';

cqlsh:faculty> DROP INDEX idx_telephone;
```


CQL Secondary Indexes

```
cqlsh:faculty> CREATE INDEX idx_title ON teachers(title);

cqlsh:faculty> SELECT firstname, lastname FROM teachers WHERE title CONTAINS
'lecturer';

cqlsh:faculty> DROP INDEX idx_title;


cqlsh:faculty> CREATE INDEX idx_title_key ON teachers(keys(title));

cqlsh:faculty> SELECT firstname, lastname FROM teachers WHERE title CONTAINS KEY
'academic' ALLOW FILTERING;

cqlsh:faculty> DROP INDEX idx_title_key;
```


CQL Secondary Indexes

- SASI: A New Secondary Index Implementation
  - The Cassandra 2.4 release included an alternative implementation of secondary indexes known as the SSTable Attached Secondary Index (SASI).
  - SASI was developed by Apple and released as an open source implementation of Cassandra's secondary index API.
  - As the name implies, SASI indexes are calculated and stored as part of each SSTable file, differing from the original Cassandra implementation, which stores indexes in separate, "hidden" tables.
  - Enable it in Cassandra.yaml: sasi_indexes_enabled: false

CQL Secondary Indexes

- SASI: A New Secondary Index Implementation
  - The SASI implementation exists alongside traditional secondary indexes.
  - SASI indexes do offer functionality beyond the traditional secondary index implementation, such as the ability to do inequality (greater than or less than) searches on indexed columns.
  - You can also use the new CQL LIKE keyword to do text searches against indexed columns.
  - While SASI indexes do perform better than traditional indexes by eliminating the need to read from additional tables, they still require reads from a greater number of nodes than a denormalized design.

CQL Secondary Indexes

- SASI: A New Secondary Index Implementation

```
cqlsh:faculty> CREATE CUSTOM index idx_lastname_sasi ON
teachers(lastname) USING 'org.apache.cassandra.index.sasi.SASIIndex';


cqlsh:faculty> select firstname, lastname from teachers where lastname like
'Tr%';

cqlsh:faculty> drop index idx_lastname_sasi;
```


CQL Aggregation

- Built-in functions and aggregates
  - COUNT
    - The COUNT function is used to count the number of rows returned by a query.
    - This command can also be used to count the number of non-null values of a specified column.
  - MIN and MAX
    - The MIN and MAX functions can be used to compute the minimum and maximum value returned by a query for a given column.

CQL Aggregation

- Built-in functions and aggregates
  - SUM
    - The sum function can be used to sum up all the values returned by a query for a given column.
  - AVG
    - The avg function can be used to compute the average of all the values returned by a query for a given column.

CQL Aggregation

- Built-in functions and aggregates

```
cqlsh:faculty> SELECT COUNT(firstname) FROM teachers;

cqlsh:faculty> SELECT MIN(age), MAX(age) FROM teachers;

cqlsh:faculty> SELECT SUM(age) FROM teachers;

cqlsh:faculty> SELECT AVG(age) FROM teachers;
```


CQL UDFs

- User defined functions
  - UDFs are functions that are applied on Cassandra nodes to stored data as part of query processing.
  - Before using UDFs in your cluster, they must be enabled in the cassandra.yaml file on each node:
    - Before v4: enable_user_defined_functions: true
    - Starting with v4: user_defined_functions_enabled: true
  - The 3.0 release added a security feature to run UDF code in a separate sandbox to limit the ability for a malicious function to gain unauthorized access to a node's Java runtime.

CQL UDFs

- UDF Restrictions
  - The code you can write inside a UDF is quite limited
  - It has to be a pure function so no IO, logging, external access of any kind.
  - Can't start threads
  - No dependencies

CQL UDFs

- User defined functions

```
cqlsh:faculty> CREATE FUNCTION age_to_range(input int)
        ... RETURNS NULL ON NULL INPUT
        ... RETURNS varchar
        ... LANGUAGE java AS '
        ...   if (input <= 35) return "young";
        ...   else if (input> 55) return "old";
        ...   else return "mature";
        ... ';

cqlsh:faculty> SELECT firstname, lastname, age_to_range(age) FROM
teachers;
```


CQL UDFs

- Custom aggregates allow you to reduce a query that returns many rows into a single value
- These are called user defined aggregation (UDA)
- For UDAs we need to:
  - Define a function that will be executed, sequentially, for every row, passing some state along
  - Initial state for the first row
  - An optional function that will take in the final state and produce a value

CQL UDFs

- Custom aggregates State function – average:

```
cqlsh:faculty> CREATE OR REPLACE FUNCTION avgState ( state
tuple<int,bigint>, val int ) CALLED
        ... ON NULL INPUT RETURNS tuple<int,bigint> LANGUAGE java AS '
        ...  if (val != null) {
        ...    state.setInt(0, state.getInt(0)+1);
        ...    state.setLong(1,state.getLong(1)+val.intValue());
        ...  }
        ...  return state;
        ... ';
```


CQL UDFs

- Custom aggregates Final function – average:

```
cqlsh:faculty> CREATE OR REPLACE FUNCTION avgFinal ( state
tuple<int,bigint> ) CALLED ON
        ... NULL INPUT RETURNS double LANGUAGE java AS '
        ...  double r = 0;
        ...  if (state.getInt(0) == 0)
        ...    return null;
        ...  r = state.getLong(1);
        ...  r /= state.getInt(0);
        ...  return Double.valueOf(r);
        ... ';
```


CQL UDFs

- Custom aggregates creating the aggregate – average

```
cqlsh:faculty> CREATE AGGREGATE IF NOT EXISTS average ( int )
        ...  SFUNC avgState
        ...  STYPE tuple<int,bigint>
        ...  FINALFUNC avgFinal
        ...  INITCOND (0,0);

cqlsh:faculty> SELECT average(age) FROM teachers;
```

CQL UDFs

- Custom aggregation to simulate – unique

```
cqlsh:faculty> CREATE OR REPLACE FUNCTION uniq(state set<int>, val int)
           ... CALLED ON NULL INPUT RETURNS set<int> LANGUAGE java
           ... AS 'state.add(val); return state;';

cqlsh:faculty> CREATE OR REPLACE AGGREGATE uniq(int)
           ... SFUNC uniq STYPE set<int> INITCOND {};

cqlsh:faculty> SELECT uniq(age) FROM teachers;
```


CQL UDFs

- Custom aggregates simulate group by

```
cqlsh:faculty> CREATE OR REPLACE FUNCTION state_group_and_max(state map<int, int>, type
int, amount int)
           ... CALLED ON NULL INPUT
           ... RETURNS map<int, int>
           ... LANGUAGE java AS '
           ...   Integer val = (Integer) state.get(type);
           ...   if (val == null) val = amount;
           ...   else val = Math.max(val, amount);
           ...   state.put(type, val);
           ...   return state;
           ... ';

cqlsh:faculty> CREATE OR REPLACE AGGREGATE state_group_and_max(int, int)
           ... SFUNC state_group_and_max
           ... STYPE map<int, int>
           ... INITCOND {};

cqlsh:faculty> SELECT state_group_and_max(age, id) FROM teachers;
```